CSE/PC/B/S/314 — Computer Networks Lab

Assignment 1

Error Detection Module — Checksum & CRC

Design and implementation of an error detection module evaluating the Internet Checksum and Cyclic Redundancy Check (CRC-8/10/16/32) over a TCP sender-receiver system.

Ritam Ghosh

Section: A2

Roll Number: 002410501040

Language: C++ (g++ -std=c++14) | Platform: macOS / Linux (POSIX sockets)

1. Objective

The objective of this assignment is to design and implement an error detection module that evaluates two fundamental schemes used in computer networks:

- Internet Checksum — a 16-bit one's complement sum, as defined in RFC 1071 and used in IP, TCP, and UDP headers.
- Cyclic Redundancy Check (CRC) — polynomial-based remainder detection, evaluated for four standard generator polynomials: CRC-8 (0xD5), CRC-10 (0x233), CRC-16 (0x8005), and CRC-32 (0x04C11DB7).

The system consists of a Sender and a Receiver communicating over TCP sockets. The Sender reads a binary file, constructs Ethernet-style frames protected by each of the five detection schemes, optionally injects controlled errors, and transmits them. The Receiver parses the incoming byte stream, recomputes the detection codes, and reports whether each error was detected or missed.

Additionally, we demonstrate special cases where each scheme has a theoretical blind spot — specifically crafted errors that a scheme is mathematically guaranteed to miss — to illustrate the practical limits of each approach.

2. Programs Submitted

File	Description
Sender.cpp	Reads a binary file, constructs frames with error-detection trailers for all 5 schemes, optionally injects errors, and transmits over TCP.
Receiver.cpp	Listens on a TCP socket, parses incoming frames, recomputes the error-detection code, verifies integrity, logs results, and serves a live web UI.
ErrorDetection.h / .cpp	Contains the two core detection algorithms: computeInternetChecksum() and computeCyclicRedundancy().
ErrorInjection.h / .cpp	Contains the error injection functions for all 5 error types used in the experiments.

Compilation

```
g++ -std=c++14 Sender.cpp ErrorDetection.cpp ErrorInjection.cpp -o sender
g++ -std=c++14 Receiver.cpp ErrorDetection.cpp -o receiver
```

3. Frame Format

Each transmitted frame follows the structure of an Ethernet IEEE 802.3-style frame. The payload size is constrained between 46 bytes (minimum Ethernet payload) and 1024 bytes (within the 1500-byte Ethernet MTU).

NetworkFrameHeader (16 bytes)			
packetId (4 bytes)	payloadSize (2 bytes)	protocolId (1 byte)	reserved (1 byte)

sourceAddress (6 bytes)		targetAddress (6 bytes)		
Payload (46 – 1024 bytes)				
Trailer (1 – 4 bytes)				
Checksum 2 bytes	CRC-8 1 byte	CRC-10 2 bytes	CRC-16 2 bytes	CRC-32 4 bytes

The header is defined in both Sender.cpp and Receiver.cpp as:

```
struct NetworkFrameHeader {
    uint32_t packetId;        // Sequence number (network byte order)
    uint16_t payloadSize;     // Original payload size before padding
    uint8_t  protocolId;      // 0: Checksum, 1: CRC-8, 2: CRC-10, 3: CRC-16, 4:
CRC-32
    uint8_t  reserved;        // Encodes error type (0 = clean, 1-7 = error type)
    uint8_t  sourceAddress[6];
    uint8_t  targetAddress[6];
};
```

Header Fields

Field	Size	Description
packetId	4 bytes	Sequence number of the dataword (network byte order via htonl)
payloadSize	2 bytes	Original payload size before padding (network byte order via htons)
protocolId	1 byte	Identifies the scheme: 0=Checksum, 1=CRC-8, 2=CRC-10, 3=CRC-16, 4=CRC-32
reserved	1 byte	Encodes whether error was injected and which type
sourceAddress	6 bytes	Simulated source MAC address (00:1A:2B:3C:4D:5E)
targetAddress	6 bytes	Simulated destination MAC address (FF:EE:DD:CC:BB:AA)

The five detection schemes and their parameters are stored in a shared configuration array:

```
struct Scheme { int type; uint32_t poly; int degree; string name; };
Scheme schemes[5] = {
    {1, 0,          0,  "Checksum"},
    {2, 0xD5,       8,  "CRC-8"},
    {2, 0x233,      10, "CRC-10"},
    {2, 0x8005,     16, "CRC-16"},
    {2, 0x04C11DB7, 32, "CRC-32"}
};
```

4. Methodology

4.1 Fair Comparison Across Schemes

For each chunk of payload data (dataword), the sender constructs five separate frames — one per scheme — from the same payload. If error injection is enabled for that dataword, the same error is applied identically to all five frames. This ensures that differences in detection outcomes reflect the properties of the schemes, not differences in the injected corruption.

The order of frame transmission is randomized for each dataword to prevent any systematic bias:

```
std::vector<int> protocolOrder = {0, 1, 2, 3, 4};
std::random_device rd;
std::mt19937 g(rd());
std::shuffle(protocolOrder.begin(), protocolOrder.end(), g);
bool injectErrorThisChunk = (rand() % 2 == 1);  // 50% corruption probability
for (int i : protocolOrder) {
    // Build frame for scheme i, inject error if selected, transmit
}
```

4.2 Transmission Protocol

- Communication uses TCP (SOCK_STREAM) sockets.
- The sender reads a binary input file in 1024-byte chunks.
- Each chunk is padded to at least 46 bytes (Ethernet minimum payload).
- For each chunk, 5 frames (one per scheme) are transmitted with a 50 ms inter-frame delay.
- Each frame carries the error-detection trailer computed before error injection. Errors are injected after the trailer is appended, simulating channel noise.

A reliable sendAll() helper ensures every byte is transmitted even if send() returns a partial write:

```
static bool sendAll(int sock, const uint8_t *data, size_t size) {
    size_t sent = 0;
    while (sent < size) {
        ssize_t result = send(sock, data + sent, size - sent, 0);
        if (result <= 0) return false;
        sent += static_cast<size_t>(result);
    }
    return true;
}
```

4.3 Error Injection Strategy

Each dataword is independently selected for corruption with probability 0.5 (coin flip). When an error is injected, the same corruption is applied to all five scheme frames for that dataword. The error type is chosen once at startup (via command-line argument or interactive menu).

5. Error Injection Types

We implement 5 types of errors. The user selects the error type via a command-line argument or interactive menu.

5.1 Single-Bit Error (Choice 0)

A single random bit in the frame is flipped. This is the simplest form of corruption and models isolated bit errors caused by thermal noise.

```
void ErrorInjection::introduceRandomBitFlip(std::vector<uint8_t>& packet) {
    if (packet.empty()) return;
    int numBits = packet.size() * 8;
    if (numBits < 3) return;
    std::default_random_engine engine(
        static_cast<unsigned int>(
            std::chrono::system_clock::now().time_since_epoch().count()));
    std::uniform_int_distribution<int> dist(0, numBits - 1);
    int target = dist(engine);
    packet[target / 8] ^= (1 << (target % 8));
}
```

Expected detection: All five schemes should detect single-bit errors with 100% probability. A single bit flip changes exactly one term in the checksum sum, and it produces an error polynomial x^k which is never divisible by any standard CRC generator (since all generators have at least 2 terms).

5.2 Two Isolated Single-Bit Errors (Choice 1)

Two bits at non-adjacent positions (separated by more than 1 bit) are independently flipped. This models two separate noise events occurring on the same frame.

```
void ErrorInjection::introduceDoubleBitFlip(std::vector<uint8_t>& packet) {
    if (packet.empty()) return;
    int numBits = packet.size() * 8;
    if (numBits < 3) return;
    std::uniform_int_distribution<int> dist(0, numBits - 1);
    int firstTarget = dist(engine);
    int secondTarget; bool found = false;
```

```
    while (!found) {
        secondTarget = dist(engine);
        if (std::abs(firstTarget - secondTarget) > 1) found = true;
    }
    packet[firstTarget / 8] ^= (1 << (firstTarget % 8));
    packet[secondTarget / 8] ^= (1 << (secondTarget % 8));
}
```

Expected detection: Both schemes should detect this. For CRC, the error polynomial x^a + x^b is not divisible by any well-chosen generator with ≥ 3 terms. For Checksum, two independent bit flips are very unlikely to cancel in one's complement arithmetic (they would need to be in complementary positions of two different 16-bit words).

5.3 Odd Number of Errors (Choice 2)

An odd number of bits (3, 5, or 7, selected randomly) are flipped at randomly chosen distinct positions.

```
void ErrorInjection::introduceOddAnomalies(std::vector<uint8_t>& packet) {
    int numBits = packet.size() * 8;
    std::uniform_int_distribution<int> distOdd(0, 2);
    int anomalyCount = 3 + (distOdd(engine) * 2);  // 3, 5, or 7
    std::unordered_set<int> mutatedBits;
    std::uniform_int_distribution<int> bitDist(0, numBits - 1);
    while (mutatedBits.size() < (size_t)anomalyCount)
        mutatedBits.insert(bitDist(engine));
    for (int target : mutatedBits)
        packet[target / 8] ^= (1 << (target % 8));
}
```

Expected detection: Any scheme with (x+1) as a factor of the generator polynomial detects all odd-weight errors, since E(1) = 1 for any polynomial with an odd number of terms, and (x+1) divides E(x) only if E(1) = 0. The Internet Checksum also detects odd-weight errors because a single bit flip within any 16-bit word changes the sum.

5.4 Burst Errors (Choice 3)

A contiguous burst of 3–8 bits is flipped starting at a random position. This models electromagnetic interference that corrupts a short run of consecutive bits — the most common error pattern in real-world channels.

```
void ErrorInjection::introduceBurstNoise(std::vector<uint8_t>& packet, int
noiseLength) {
    if (packet.empty() || noiseLength <= 1) return;
```

```
    int numBits = packet.size() * 8;
    if (noiseLength > numBits) noiseLength = numBits;
    std::uniform_int_distribution<int> startDist(0, numBits - noiseLength);
    int startPos = startDist(engine);
    int k = 0;
    while (k < noiseLength) {
        int target = startPos + k;
        packet[target / 8] ^= (1 << (target % 8));
        k++;
    }
}
```

The burst length is randomized in Sender.cpp: case 3: ErrorInjection::introduceBurstNoise(frame, 3 + (rand() % 6)); break;

Expected detection: CRC-n is guaranteed to detect all burst errors of length ≤ n. Since our bursts are 3–8 bits and even CRC-8 has degree 8, all four CRC variants should detect them. The Checksum also detects short bursts because the corrupted bits typically do not cancel within the one's complement sum.

5.5 Checksum Collision Error (Choice 4)

This error specifically targets the algebraic structure of the Internet Checksum. It finds two 16-bit words in the frame that differ at some bit position k and flips bit k in both words. Because one word gains +2^k and the other gains -2^k in one's complement, the sum is unchanged.

```
void ErrorInjection::introduceChecksumCollision(std::vector<uint8_t>& packet) {
    size_t dataSize = packet.size() - 2;
    size_t numWords = dataSize / 2;
    for (int attempt = 0; attempt < 200; ++attempt) {
        size_t firstWord = wordDist(engine), secondWord = wordDist(engine);
        if (firstWord == secondWord) continue;
        int bitPos = bitDist(engine);
        // read wordA, wordB; if bitA != bitB, flip that bit in both words
        if (bitA != bitB) { flipWordBit(firstWord, bitPos);
flipWordBit(secondWord, bitPos); return; }
    }
    introduceRandomBitFlip(packet);  // fallback
}
```

Expected detection: The Internet Checksum misses this error (0% detection) because the one's complement sum is preserved. All four CRC variants detect it because the error polynomial (two bits flipped at specific positions) is not divisible by the CRC generators.

6. Detection Algorithms

6.1 Internet Checksum (16-bit One's Complement)

The checksum divides the data into 16-bit words, sums them using one's complement addition (with end-around carry), and takes the bitwise complement. This is the algorithm used in IP, TCP, and UDP headers.

```
uint16_t ErrorDetection::computeInternetChecksum(std::vector<uint8_t>& payload) {
    uint32_t acc = 0; size_t length = payload.size(), index = 0;
    while (length > 1) {
        uint16_t chunk = (payload[index] << 8) | payload[index + 1];
        acc += chunk; index += 2; length -= 2;
    }
    if (length > 0) acc += (payload.back() << 8);
    while (acc >> 16) acc = (acc & 0xFFFF) + (acc >> 16);
    return static_cast<uint16_t>(~acc);
}
```

How it works

- The data is treated as a sequence of 16-bit words.
- All words are summed using a 32-bit accumulator.
- Any carry beyond 16 bits is folded back — the “end-around carry” that makes the arithmetic one's complement.
- The bitwise NOT (~) of the final sum is the checksum.

Detection guarantees

- Detects all single-bit errors.
- Detects all errors with an odd number of bit flips.
- Cannot detect errors that preserve the algebraic sum of 16-bit words (e.g., swapping two words, or complementary flips in two words).

6.2 Cyclic Redundancy Check (CRC)

CRC treats the entire message as a polynomial M(x) over GF(2) and computes the remainder R(x) = x^n × M(x) mod G(x), where G(x) is the generator polynomial of degree n. Our implementation uses a bit-by-bit shift register approach:

```cpp
uint32_t ErrorDetection::computeCyclicRedundancy(std::vector<uint8_t>& payload,
                                                 uint32_t poly, int bitLen) {
    uint32_t registerVal = 0;
    uint32_t boundary = (bitLen == 32) ? 0xFFFFFFFF : (ULL << bitLen) - 1;
    for (size_t k = 0; k < payload.size(); ++k) {
        uint8_t currentByte = payload[k];
        for (int b = 7; b >= 0; --b) {
            uint8_t extractedBit = (currentByte >> b) & 1;
            uint8_t topBit = (registerVal >> (bitLen - 1)) & 1;
            registerVal = (registerVal << 1) | extractedBit;
            if (topBit) registerVal ^= poly;
        }
    }
    int padCount = bitLen;
    while (padCount > 0) {
        uint8_t topBit = (registerVal >> (bitLen - 1)) & 1;
        registerVal = (registerVal << 1);
        if (topBit) registerVal ^= poly;
        padCount--;
    }
    return registerVal & boundary;
}
```

How it works

- A shift register of width n (the polynomial degree) is initialized to 0.
- Each message bit is shifted into the register from the right.
- If the bit shifted out from the left (MSB) is 1, the register is XORed with the generator polynomial — the polynomial division step.
- After all message bits, n zero bits are shifted through (augmentation), producing the final remainder.
- The remainder is masked to n bits.

Generator Polynomials Used

Scheme	Polynomial	Hex	Degree	Trailer
CRC-8	x^8+x^7+x^6+x^4+x^2+1	0xD5	8	1 byte
CRC-10	x^10+x^9+x^5+x^4+x+1	0x233	10	2 bytes
CRC-16	x^16+x^15+x^2+1	0x8005	16	2 bytes
CRC-32	x^32+x^26+x^23+...+1	0x04C11DB7	32	4 bytes

Detection guarantees (for CRC-n with a well-chosen generator)

- Detects all single-bit errors.
- Detects all double-bit errors (if the generator has at least 3 terms).
- Detects all odd-weight errors (if (x+1) is a factor of G(x)).
- Detects all burst errors of length ≤ n.
- Cannot detect errors whose error polynomial E(x) is divisible by G(x).

7. Sender Program

7.1 Operation

The Sender reads a binary input file and transmits it as a sequence of protected frames.

File reading

The file is read in chunks of up to 1024 bytes. Short chunks are padded to 46 bytes (minimum Ethernet payload):

```
vector<uint8_t> buffer(MAX_PAYLOAD_SIZE);
uint32_t sequenceCounter = 0;
while (file.read((char*)buffer.data(), MAX_PAYLOAD_SIZE) || file.gcount() > 0) {
    size_t bytesRead = file.gcount();
    vector<uint8_t> payload(buffer.begin(), buffer.begin() + bytesRead);
    if (payload.size() < MIN_PAYLOAD_SIZE)
        payload.resize(MIN_PAYLOAD_SIZE, 0x00);  // Pad to 46 bytes
```

Frame construction

For each chunk, 5 frames are built (one per scheme). The header is populated, the payload is appended, and the appropriate trailer is computed and appended:

```
NetworkFrameHeader header;
memcpy(header.sourceAddress, sourceMac, 6);
memcpy(header.targetAddress, destinationMac, 6);
header.payloadSize = htons((uint16_t)bytesRead);
header.packetId = htonl(sequenceCounter);
header.protocolId = i;
header.reserved = injectErrorThisChunk ? (errorChoice + 1) : 0;
vector<uint8_t> frame;
frame.insert(frame.end(), hdrBytes, hdrBytes + sizeof(NetworkFrameHeader));
frame.insert(frame.end(), payload.begin(), payload.end());
```

Trailer computation

The checksum or CRC is computed over the header+payload and appended:

```
if (schemes[i].type == 1) {
    uint16_t checksum = ErrorDetection::computeInternetChecksum(frame);
    frame.push_back((checksum >> 8) & 0xFF); frame.push_back(checksum & 0xFF);
} else {
    uint32_t crc = ErrorDetection::computeCyclicRedundancy(frame, schemes[i].poly,
schemes[i].degree);
    // push 1, 2, or 4 bytes of crc depending on degree
}
```

Error injection

If this dataword is selected for corruption, the chosen error is applied after the trailer (simulating channel noise):

```
if (injectErrorThisChunk) {
    switch (errorChoice) {
    case 0: ErrorInjection::introduceRandomBitFlip(frame); break;
    case 1: ErrorInjection::introduceDoubleBitFlip(frame); break;
    case 2: ErrorInjection::introduceOddAnomalies(frame); break;
    case 3: ErrorInjection::introduceBurstNoise(frame, 3 + (rand() % 6)); break;
    case 4: ErrorInjection::introduceChecksumCollision(frame); break;
    }
}
```

Transmission

The frame is sent over TCP and the status is logged:

```
if (!sendAll(sock, frame.data(), frame.size())) {
    cerr << "Transmission failed.\n"; close(sock); return -1;
}
cout << "[SENT] SeqNo: " << sequenceCounter << " | Scheme: " << schemes[i].name
     << " | Size: " << frame.size() << " bytes | "
     << (injectErrorThisChunk ? "[CORRUPTED]" : "[CLEAN]") << "\n";
```

7.2 Command-Line Usage

```
./sender <filename> [receiver-ip] [port] [errorChoice]
```

- filename — path to the input file (required)
- receiver-ip — defaults to 127.0.0.1
- port — defaults to 8080

- errorChoice — 0 to 4 (if omitted, an interactive menu is shown)

8. Receiver Program

8.1 Operation

The Receiver listens for incoming frames on a TCP socket and verifies the integrity of each one.

Stream parsing

Since TCP is a byte stream, frames may arrive split or concatenated. The receiver accumulates bytes in a pending buffer and extracts complete frames:

```
vector<uint8_t> pending;
while ((bytesRead = recv(new_socket, buffer.data(), buffer.size(), 0)) > 0) {
    pending.insert(pending.end(), buffer.begin(), buffer.begin() + bytesRead);
    while (pending.size() >= sizeof(NetworkFrameHeader)) {
        NetworkFrameHeader peekHeader;
        memcpy(&peekHeader, pending.data(), sizeof(NetworkFrameHeader));
        size_t frameSize;
        if (!computeFrameSize(peekHeader, frameSize)) break; // resync
        if (pending.size() < frameSize) break;
        vector<uint8_t> frame(pending.begin(), pending.begin() + frameSize);
        pending.erase(pending.begin(), pending.begin() + frameSize);
```

Frame size computation

Uses the protocolId to determine the trailer length:

```
static bool computeFrameSize(const NetworkFrameHeader &header, size_t &frameSize)
{
    uint8_t protocolId = header.protocolId;
    if (!isValidProtocolId(protocolId)) return false;
    size_t payloadSize = max<size_t>(ntohs(header.payloadSize), MIN_PAYLOAD_SIZE);
    if (payloadSize > MAX_FRAME_SIZE) return false;
    size_t trailerSize = (protocolId == 0) ? 2 : (protocolId == 1 ? 1 :
(protocolId == 4 ? 4 : 2));
    frameSize = sizeof(NetworkFrameHeader) + payloadSize + trailerSize;
    return (frameSize <= MAX_FRAME_SIZE);
}
```

Integrity verification

The trailer is stripped, the detection code is recomputed, and the two values are compared. Execution time is measured using std::chrono::high_resolution_clock:

```
if (currentScheme.type == 1) {
    uint8_t chkLow = frame.back(); frame.pop_back();
    uint8_t chkHigh = frame.back(); frame.pop_back();
    uint16_t rxChecksum = (chkHigh << 8) | chkLow;
    auto start = chrono::high_resolution_clock::now();
    uint16_t calcChecksum = ErrorDetection::computeInternetChecksum(frame);
    auto end = chrono::high_resolution_clock::now();
    executionTimes[protocolId] = chrono::duration_cast<chrono::microseconds>(end-
start).count();
    detectedErrors[protocolId] = (rxChecksum != calcChecksum);
} else {
    // CRC verification: strip CRC bytes, recompute, compare, time it
}
```

Logging

Each frame's result is logged to the console and appended to a CSV file:

```
if (detectedErrors[protocolId]) {
    logLine("[REJECTED] SeqNo: " + to_string(currentSeqNo) + " | Scheme: " +
currentScheme.name + " caught an error!");
} else {
    logLine("[ACCEPTED] SeqNo: " + to_string(currentSeqNo) + " | Scheme: " +
currentScheme.name + " received clean (or missed error).");
}
```

8.2 Live Web Dashboard

The Receiver also runs a secondary HTTP web server (on port 8081 by default) using Server-Sent Events (SSE). This allows monitoring the receiver output in real-time from a browser, even from another device on the same network:

```
signal(SIGPIPE, SIG_IGN);
thread(runWebServer, webPort).detach();
```

The web UI displays a dark-themed, monospace-font log that auto-scrolls as new frames are received.

8.3 Command-Line Usage

```
./receiver [port] [web-port] [bind-address]
```

9. Empirical Results

All experiments were conducted using a 10 KB random binary input file, yielding 10 datawords per run. Each error type was tested in a separate run across all 5 schemes simultaneously.

9.1 Detection Rates

The following chart shows the detection rate (%) for each scheme across all 5 error types:

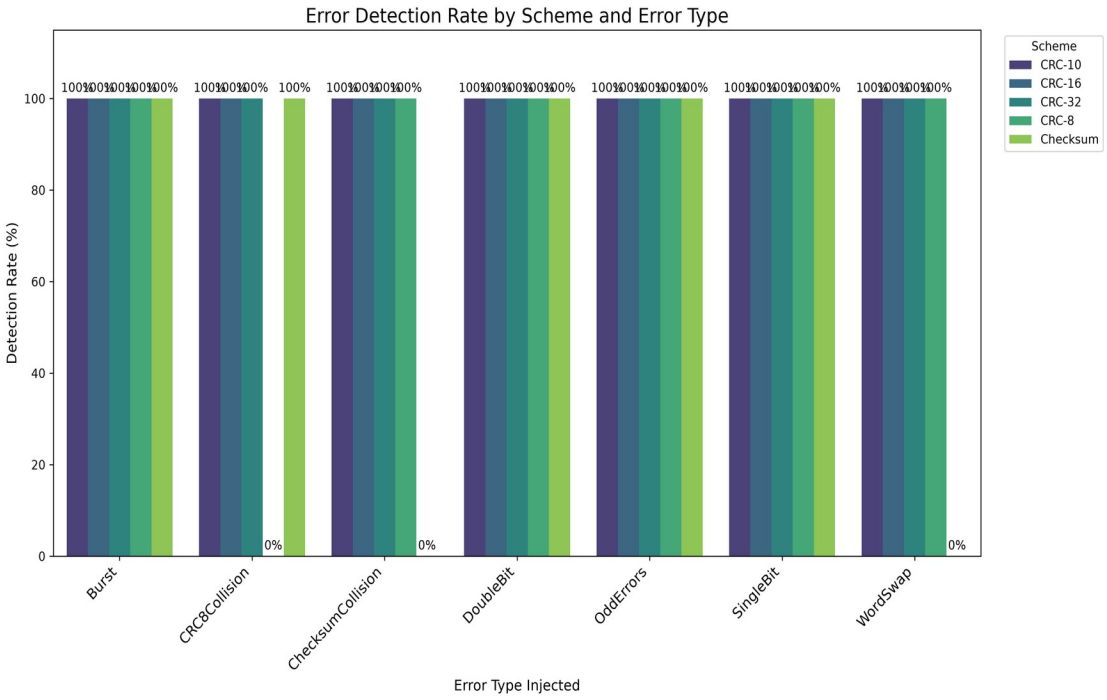

Figure 1: Error Detection Rate by Scheme and Error Type

Results summary

Error Type	Checksum	CRC-8	CRC-10	CRC-16	CRC-32
Single-Bit	100%	100%	100%	100%	100%
Double-Bit	100%	100%	100%	100%	100%
Odd Errors	100%	100%	100%	100%	100%
Burst	100%	100%	100%	100%	100%
Checksum Collision	0%	100%	100%	100%	100%

Key observations

- For the first four error types (standard noise models), all five schemes achieve 100% detection. Both Checksum and CRC are fully reliable against these common corruption patterns.
- The Checksum Collision error exploits the algebraic weakness of one's complement addition: Checksum fails to detect it (0%) while all CRC variants catch it (100%).

9.2 Execution Times

The following chart shows the mean execution time for the detection computation on each scheme:

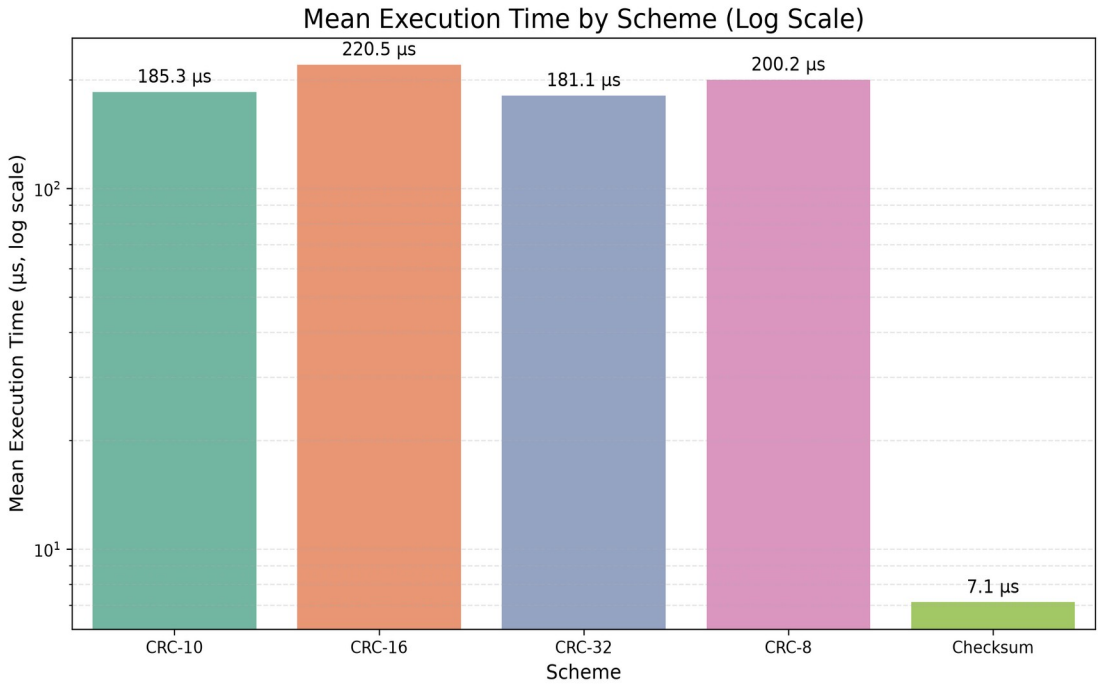

Figure 2: Mean Execution Time by Scheme (Log Scale)

Key observations

- The Internet Checksum is approximately 20–30x faster than all four CRC variants.
- This is expected: the checksum performs simple 16-bit additions (one per 2 bytes of data), while the software CRC processes every single bit through a shift register with a conditional XOR.
- Among CRC variants, execution times are similar because the inner loop structure is identical — only the register width changes slightly.
- This speed difference explains why transport protocols (TCP/UDP) use the faster checksum despite its weaker detection: at network line rates, the computational savings are significant, and CRC-32 at the Ethernet layer already provides stronger per-frame protection.

10. Special Cases: Where Detection Schemes Fail

Beyond the 5 standard error types, we implemented two additional targeted attacks to demonstrate that each family of schemes has theoretical blind spots. These are not part of the main error suite but serve as educational demonstrations of the mathematical limitations.

10.1 Word Swap — Checksum Fails, CRC Succeeds

The Internet Checksum is fundamentally a commutative sum: Checksum = ~(w1 + w2 + w3 + ... + wn). Since addition is commutative, swapping any two 16-bit words wi and wj does not change the sum. The data has been corrupted (bytes are reordered), but the checksum is identical — a false negative.

We implemented this as introduceWordSwap(), which picks two random 16-bit words in the payload and swaps them:

```
void ErrorInjection::introduceWordSwap(std::vector<uint8_t>& packet) {
    size_t headerSize = 16;
    size_t dataSize = packet.size() - headerSize - 4;
    size_t numWords = dataSize / 2;
    size_t w1 = wordDist(engine), w2 = wordDist(engine);
    while (w1 == w2) w2 = wordDist(engine);
    size_t idx1 = headerSize + w1 * 2, idx2 = headerSize + w2 * 2;
    std::swap(packet[idx1], packet[idx2]);
    std::swap(packet[idx1+1], packet[idx2+1]);
}
```

Result: Across all Word Swap trials, Checksum reported 0% detection while all CRC variants reported 100% detection. CRC is position-sensitive — the polynomial remainder depends on where each bit appears in the message, so moving data between positions always changes the remainder.

10.2 CRC-8 Polynomial Collision — CRC-8 Fails, Checksum Succeeds

CRC computes R(x) = M(x) mod G(x). If we inject an error polynomial E(x) that is an exact multiple of the generator G(x), then (M(x) + E(x)) mod G(x) = R(x) + 0 = R(x) — the CRC remainder is unchanged. We implemented this for CRC-8 by XORing the 9-bit generator polynomial G(x) = 0x1D5 (111010101 in binary) directly into the data at a random byte-aligned position:

```
void ErrorInjection::introduceCRC8Collision(std::vector<uint8_t>& packet) {
    size_t headerSize = 16;
    size_t pos = posDist(engine);
    // 9-bit CRC-8 generator 111010101, byte-aligned: 0xEA, 0x80
    packet[pos] ^= 0xEA;
    packet[pos+1] ^= 0x80;
}
```

Result: Across all CRC-8 Collision trials, CRC-8 reported 0% detection, while Checksum and the other CRC variants (CRC-10, CRC-16, CRC-32) reported 100% detection. The higher-degree CRC generators are not divisible by the CRC-8 generator, so the same error polynomial is not invisible to them. The Checksum detects it because 0xEA80 has a non-zero one's complement sum.

10.3 Why This Matters

These special cases demonstrate that:

- No single scheme is universally superior. Checksum misses word-order errors; CRC-8 misses polynomial multiples of its generator.
- Complementary use is the real-world solution. Modern networks layer both schemes: CRC-32 at the data-link layer (Ethernet FCS) for strong per-frame protection, and the Internet Checksum at the transport layer (TCP/UDP) as a lightweight end-to-end check.
- Higher-degree CRCs are exponentially harder to fool. CRC-32 was never defeated by any of our error types — the probability of a random error being divisible by a 32-degree polynomial is approximately 1 in 2^32 (about 1 in 4 billion).

11. Summary and Conclusion

11.1 Summary

We implemented a complete error detection evaluation system consisting of:

- A Sender that constructs Ethernet-style frames with 5 different error-detection trailers (Internet Checksum, CRC-8, CRC-10, CRC-16, CRC-32) and injects 5 classes of controlled errors.
- A Receiver that parses the TCP byte stream, recomputes the detection codes, measures execution time, logs metrics, and provides a live web dashboard.
- Additional targeted collision demonstrations showing specific failure cases for both Checksum and CRC-8.

11.2 Key Findings

For standard noise models (single-bit, double-bit, odd-count, and burst errors), both the Internet Checksum and all four CRC variants achieve 100% detection. In these common scenarios, both schemes are fully reliable.

The Internet Checksum has a fundamental algebraic weakness: because it is based on commutative one's complement addition, it cannot detect any error that preserves the multiset sum of 16-bit words. We demonstrated this with the Checksum Collision error (complementary bit flips) and the Word Swap (reordering words) — both achieved 0% detection by Checksum while all CRC variants detected them at 100%.

CRC is not invulnerable: we demonstrated that CRC-8 can be defeated by injecting an error polynomial that is a multiple of its generator. The CRC-8 Collision error evaded CRC-8 at 0% detection, while the Checksum and higher-degree CRCs caught it at 100%.

Higher-degree CRCs are more robust: CRC-10, CRC-16, and CRC-32 were never fooled by any of our error types. Their longer generator polynomials make it exponentially harder to construct an error polynomial that divides evenly. CRC-32, used in Ethernet, provides the strongest protection.

Checksum is significantly faster: the Internet Checksum is approximately 20–30x faster than the software CRC implementations, reflecting its simpler computation (word-level additions vs. bit-by-bit polynomial division). This explains why UDP and IP headers use checksums despite their weaker detection properties — the computational savings matter at line rate, and the Ethernet CRC-32 already guards the frame at a lower layer.

11.3 Conclusion

The Internet Checksum and CRC represent two points on the speed vs. robustness tradeoff in error detection.

Checksum is fast and simple but has known algebraic blind spots (word reordering, compensating bit flips). It is suitable for protocol headers where speed matters and additional layers provide stronger protection.

CRC with a well-chosen polynomial provides mathematically stronger guarantees — detecting all bursts up to degree n, all single and double errors, and all odd-weight errors — at the cost of higher computational overhead. CRC-32 is the standard for Ethernet frame check sequences (FCS).

In practice, modern networks use both: CRC-32 at the data-link layer (Ethernet) for strong per-frame protection, and the Internet Checksum at the transport layer (TCP/UDP) as a lightweight end-to-end integrity check. Our experiments confirm these design choices empirically.